

SQL

Database Create

```
CREATE DATABASE DB-name;
```

Table Create

```
CREATE TABLE Tb-name (  
    col-name1 datatype(size),  
    col-name2 datatype(size), ...);
```

Example:

```
CREATE TABLE student
```

```
( Name varchar(30),
```

```
  student_ID int(8),
```

```
  NID int(8),
```

```
  Intake int(2),
```

```
  CGPA float(3,2));
```

Value insert in table / row

```
INSERT INTO Tbl-name values (value1, value2, value3, ...)
```

Example:

```
INSERT INTO student ("Kaniz", 154, 1001, 53, 7.93)
```

! [Double Quotes for Varichare

float (3,2)

→ Num of digits after the decimal point

→ Total num of digits (before + after decimal point)

Update NULL values / values

```
UPDATE tb_name  
SET col_name = "new-value"  
WHERE condition ;
```

Example:

```
UPDATE faculty  
SET short_code = "BM"  
WHERE Name = "Baymax";
```

Add Column

```
ALTER TABLE tb_name  
ADD COLUMN col_name (datatype(size));
```

Example:

```
ALTER TABLE student  
ADD COLUMN Age int(2);
```

Delete column

```
ALTER TABLE tb_name  
DROP COLUMN col_name ;
```

Example:

```
ALTER TABLE student  
DROP COLUMN NID;
```

ALTER TABLE

- to change structure of a table

- add column
- delete column
- rename column
- change datatype of a column

Project column info

```
SELECT col-name  
FROM tb-name;
```

Example: Find the names of all courses in the course relation.

```
SELECT course_title  
FROM course
```

R.A: π course_title (course)

Project column with condition

```
SELECT col-name  
FROM tb-name  
WHERE condition;
```

Example: Select Names of all teachers who
has conducted in summer 2019.

```
SELECT Name
```

```
FROM faculty
```

```
WHERE conduct_semester = "Summer 2019";
```

R.A:

π Name (σ conduct_semester = "Summer 2019" (faculty))

Relational Algebra

- Select (σ)
- Project (π)
- Union ($\cup$)
- Intersection ($\cap$)
- Set difference ($-$)
- Cartesian product ($\times$)
- AND ($\wedge$)
- OR ($\vee$)

σ vs π

π (selected col, All rows)



Kaniz Fatema
20245103154

Add column after existing column

```
ALTER TABLE Tb-name  
ADD COLUMN new-col datatype (size)  
AFTER existing-col-name ;
```

Example: Add a new column 'Age' after the column 'Name'

```
ALTER TABLE student  
ADD COLUMN Age int (2)  
AFTER Name ;
```

Project value in Alphabetic Order

```
SELECT col-name  
FROM Tb-name  
ORDER BY col-name ASC ;
```

! **ASC** - Ascending Order
(A, B, C, D, ...)
DESC - Descending Order
(Z, Y, X, W, ...)

Example: Find Name and CGPA of each student order by their CGPA

```
SELECT Name, CGPA  
FROM student
```

```
ORDER BY CGPA ASC ;
```

in Ascending Order

AND condition

SELECT $\frac{*}{\text{col-name}}$

FROM $\frac{*}{\text{tb-name}}$

WHERE $\frac{*}{\text{condition 1}}$ AND $\frac{*}{\text{condition 2}}$;

! AND : $\wedge$
OR : $\vee$

Example: Find the name of all students, whose age > 22 and CGPA > 7.23 ;
 सभी name लैवतल शर

SELECT Name

FROM student

WHERE Age > 22 AND CGPA > 7.23 ;

R.A: $\pi_{\text{Name}} (\sigma_{\text{Age} > 22 \wedge \text{CGPA} > 7.23} (\text{student}))$

Multiple OR condition

SELECT $\frac{*}{\text{col-name}}$

FROM $\frac{*}{\text{tb-name}}$

WHERE $\boxed{\text{IN}}$ ($\frac{*}{\text{condition}}$);

! IN - show info containing giving value
NOT IN - show info that doesn't contain given value

Example: Find the id of all these students whose CGPA is 4.30 or 4.50 or 4.96.

SELECT student-ID

FROM student

WHERE IN (4.30, 4.50, 4.96);

R.A: $\pi_{\text{student-ID}} (\sigma_{\text{CGPA} = 4.30 \vee \text{CGPA} = 4.50 \vee \text{CGPA} = 4.96} (\text{student}))$

Max, Min, Avg

```
SELECT  
  MAX (*col-name)  
  MIN (*col-name)  
  AVG (*col-name)  
FROM *tb-name ;
```

Example: Find the largest, minimum and avg. account balance in "Account" relation

```
SELECT  
  MAX (balance)  
  MIN (balance)  
  AVG (balance)  
FROM Account ;
```

Num of Tuples

```
SELECT  
  COUNT (*col-name)  
FROM *tb-name ;
```

Example: Find the total number of customer from "Customers" Relation

```
SELECT  
  COUNT (customer_name)  
FROM customers ;
```

R.A:

G1 count (customer_name)

Between operation

```
SELECT col_name  
FROM tb_name  
WHERE condition 1 BETWEEN condition 2 ;
```

Example: Find the loan number of those loans with loan amount between 400 and 800.

```
SELECT loan_number  
FROM loan  
WHERE amount BETWEEN 400 AND 800 ;
```

RA: $\pi_{\text{loan_number}} (\sigma_{\text{amount} \geq 400 \wedge \text{amount} \leq 800} (\text{loan}))$

String function

```
SELECT col_name  
FROM tb_name  
WHERE col_name LIKE Pattern ;
```

!

- "A%" starts with 'A'
- "%.Z" ends with 'Z'
- "%.cat%" contains 'cat' anywhere
- "_ _ n %" third letter 'n'

Example: Find the names of all customers whose name start with "G".

```
SELECT customer_name  
FROM customer  
WHERE customer_name LIKE "G%";
```

RA: $\pi_{\text{customer_name}} (\sigma_{\text{customer_name} = "G\%"} (\text{customer}))$

Length of values

```
SELECT col_name,  
LENGTH (col_name)  
FROM tb_name;
```

Example: Find the length of the names of all customers from Customer

```
SELECT (customer_name)  
LENGTH (customer_name)  
FROM customer;
```

Substr (Extract substring)

```
SELECT col_name,  
SUBSTR (col_name, start, length)  
FROM tb_name;
```

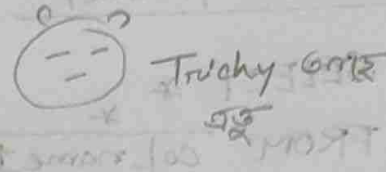
starting position 1 for first char
num of lengths in sub string

Example: Find 1st three characters of each customer name from Customer.

```
SELECT customer_name  
SUBSTR (customer_name, 1, 3)  
FROM customer;
```


Group by

```
SELECT col-name  
FROM tb-name  
GROUP BY col-name;
```



Example: Find the number of customers from all cities in the customer relation.

```
SELECT customer-cities  
COUNT (customer-name)  
FROM customer  
GROUP BY customer-cities;
```

Use of Having

```
SELECT col-name, aggregate-function (column 2)  
FROM tb-name  
WHERE condition  
GROUP BY col-name  
HAVING aggregate-function (column 2) condition;
```

Example: Find the avg account balance of each branch whose avg account balance is greater than 500.

```
SELECT branch-name, AVG(balance)  
FROM account  
GROUP BY branch-name  
HAVING AVG(balance) > 500;
```

Cartesian Product

```
SELECT *  
FROM col_name1, col_name2
```

Example: Perform cartesian Product operation between Instructor and teachers.

```
SELECT *  
FROM Instructor, teachers
```

Inner Join:

```
SELECT col_name  
FROM tb_name1  
INNER JOIN tb_name2  
WHERE tb_name1.column = tb_name2.column
```

Example: Find those instructors who teaches any of the course

```
SELECT Name, ID  
FROM instructor  
INNER JOIN teachers  
WHERE instructor.ID = teachers.ID
```

Natural Join ($\bowtie$)

SELECT $\frac{*}{col_name}$
FROM $\frac{*}{tb_name1}$
NATURAL JOIN $\frac{*}{tb_name2}$;

Example: Find those ~~instructor~~ who teaches any of the courses

SELECT Name, ID
FROM instructor
NATURAL JOIN teaches ;

R.A: π Name, ID (Instructor $\bowtie$ teaches)

Outer Join:

1. Left outer Join ($\Leftarrow \bowtie$)
2. Right outer Join ($\bowtie \Rightarrow$)
3. Full outer Join ($\bowtie \Leftarrow \Rightarrow$)

SELECT $\frac{*}{*}$
FROM $\frac{*}{tb_name1}$
NATURAL $\frac{Left/Right/Full}{\text{OUTER JOIN}}$ $\frac{*}{tb_name2}$;

RA: π col_name1, col_name2, col_name3.. ($tb_name1 \bowtie tb_name2$)